TECHNICAL SPECIFICATION

SIMLOCK N8 TEST PLATFORM

N8 Test Controller, DL MCU Interface & MQTT Fleet Reporting

Hardware interconnect, UART protocol, device provisioning, REST API, and distributed MQTT test-fleet reporting for the SIMLOCK door-lock test system

Document Type	Technical Specification (Derived)
System	SIMLOCK Distributed Test Platform
Components Covered	N8 (ESP32-C6-DevKitC-1-N8) ↔ DL MCU, MQTT Fleet Reporting
Version	2.1
Status	Draft — for engineering review
Date	August 6, 2026
Source Basis	SIMLOCK GEEK–DL MCU Test Procedure (v1) + N8/MQTT Update Directive (v2) + Review Feedback (v2.1)

Revision History

Document Revision History

Version	Date	Summary of Changes
1.0	Aug 2, 2026	Initial release. Point-to-point interface between the Waveshare GEEK (ESP32-S3) test controller and the DL MCU. BLE + SoftAP provisioning. Standalone REST API only, no fleet reporting.
2.0	Aug 4, 2026	Replaced GEEK with the ESP32-C6-DevKitC-1-N8 dev board (full GPIO breakout, added Camera MCU connector). Redefined the system as a distributed multi-board test platform with a new MQTT Reporting Specification (Section 8) feeding a central backend and live dashboard. Simplified provisioning to SoftAP + captive portal only (BLE path removed). Significantly expanded the REST API (Section 7, ~40 endpoints across 11 groups). Added the OTA versioning scheme, update-server URL, and version-manifest format (Section 7.10). Reformatted the Requirements Traceability Matrix to a full 50-row test mapping (Section 9). Flagged the DPID command set beyond 21/22, the credential data structure, error-handling scheme, and WebSocket payload schema as Reserved/TBD pending confirmation from the DL MCU firmware team — see the callouts in Section 4.
2.1	Aug 6, 2026	Incorporated external review feedback: added implementation-guidance notes for board_id generation (§8.2), MQTT broker-address provisioning (§5.1), MQTT transport security / TLS / auth (§8.2), and OTA firmware-signing vs. MD5 (§7.10). Added clearly-labeled illustrative (non-normative) examples for the reserved error-code structure (§4.6) and WebSocket payload schema (§7.11) — marked “Not Confirmed” throughout and not a substitute for the firmware team's authoritative values. Added a hardware caution (§3.3) flagging that GPIO4 and GPIO5, used for MRDY and TXD, are ESP32-C6 strapping pins per Espressif's official documentation — identified during independent verification, not part of the original review request.

Table of Contents

1. Introduction

1.1 Purpose

This document specifies the functional behavior, hardware interconnection, communication protocol, provisioning procedure, REST API, and MQTT fleet-reporting interface of the SIMLOCK distributed test platform — built around the ESP32-C6-DevKitC-1-N8 ("N8") test controller and the DL MCU door-lock controller. It is intended for firmware engineers, hardware engineers, backend/dashboard developers, and quality assurance personnel who design, integrate, extend, or validate SIMLOCK-compatible door-lock test hardware.

1.2 Scope

This specification covers:

- The physical (5-wire UART) and logical (MRDY/SRDY handshake, frame structure) interface between the N8 board and the DL MCU.
- The N8 hardware platform, including its Camera MCU (SPI) connector, which the previous GEEK-based platform could not support.
- Device provisioning via SoftAP and captive portal, followed by MQTT broker connection.
- Functional behavior of direct lock control, PIN/RFID/fingerprint credential management, alerts and security events, backlight control, logging and status reporting, configuration parameters, system management, and firmware version cycling / OTA update.
- The REST API surface exposed locally by each N8 board.
- The MQTT topic structure, message formats, and remote-command channel used for fleet-wide test reporting to a central backend and dashboard (new in this revision).

This specification does not cover:

- Internal DL MCU or Camera MCU firmware implementation or source code.
- Mechanical door-lock hardware design.
- Backend server or dashboard implementation details beyond the MQTT/WebSocket/REST contracts they consume.
- Network infrastructure beyond the local Wi-Fi segment and the MQTT broker endpoint.

1.3 Definitions, Acronyms, and Abbreviations

Table 1 — Definitions and Acronyms

Term	Definition
N8	The ESP32-C6-DevKitC-1-N8 dev board; test controller that bridges the DL MCU (and optionally a Camera MCU) to Wi-Fi, the local REST/WebSocket API, and the MQTT broker. Replaces the earlier GEEK stick as of Version 2.0.
DL MCU	Door-Lock Microcontroller Unit — the target device under test; drives the physical lock, keypad, RFID reader, fingerprint sensor, buzzer, and backlight.

Term	Definition
Camera MCU	An image/vision module connected to the N8 over SPI (GPIO18–21). Not supported on the previous GEEK platform due to insufficient GPIO.
SIMLOCK	The local, per-board web application and REST API served directly by each N8 for on-bench control and testing.
MRDY / SRDY	Master Ready / Slave Ready — the active-low handshake pair (N8 = master, DL MCU = slave) that gates each UART transaction.
DPID	Data Point Identifier — numeric code identifying the operation encoded in a UART command frame. Only DPID 21 (Add Credential) and 22 (Delete Credential) are confirmed; see Section 4.4.
OTP	One-Time PIN — a PIN credential valid for a single successful use, then automatically rejected.
NVS	Non-Volatile Storage — flash-backed key-value storage on the ESP32 used to persist Wi-Fi credentials and firmware version state.
SoftAP	Software-enabled Access Point — a temporary Wi-Fi access point hosted by the N8 itself during provisioning.
RSSI	Received Signal Strength Indicator (Wi-Fi signal strength, in dBm).
OTA	Over-the-Air firmware update.
Hardware ID	Unique identifier assigned to an enrolled credential (PIN, RFID card, or fingerprint), used to reference it for deletion or listing.
MQTT	Message Queuing Telemetry Transport — the publish/subscribe protocol N8 boards use to report live status to a central broker (Section 8).
Broker	The central MQTT server (e.g., Mosquitto, HiveMQ, or AWS IoT Core) that receives publications from N8 boards and relays them to subscribed backend services.
QoS	MQTT Quality of Service level. This platform uses QoS 1 (“at least once” delivery) throughout.
Board ID	Unique identifier for a given N8 board (e.g., N8-001), used as the `{board_id}` placeholder in every MQTT topic.
Backend Server	Your server-side application that subscribes to MQTT topics, persists test history, and serves the web dashboard.
Web Dashboard	The fleet-wide monitoring UI, showing live pass/fail counts, alerts, and OTA status across all connected N8 boards.

1.4 Reference Basis

The requirements in Sections 5 and 6 are derived from, and remain traceable to, the original SIMLOCK GEEK–DL MCU Hardware-in-the-Loop Test Procedure, a 50-case validation procedure spanning ten functional areas. Section 9 provides the mapping between this specification and those source test cases. Sections 2, 3, 7, and 8 incorporate the N8 hardware migration and MQTT fleet-reporting design supplied for Version 2.0 of this specification.

> **RESERVED / TO BE DEFINED**
> *Four items requested for this revision were not supplied with concrete values and are explicitly flagged, rather than invented, at their point of use in Section 4 and Section 7.11: (1) the complete DPID command set beyond 21 and 22,*

(2) the byte-level credential data structure, (3) a formal error-handling / error-code scheme, and (4) the WebSocket message payload schema (only the /ws endpoint path is confirmed). Each is marked Reserved / To Be Defined and should be confirmed against the authoritative DL MCU and N8 firmware references before production use. For items (3) and (4), Version 2.1 adds a clearly-labeled illustrative example showing the shape such a structure could take — these examples are explicitly not confirmed values and should not be implemented against.

2. System Architecture

2.1 Overview

As of Version 2.0, SIMLOCK is a distributed test platform rather than a single point-to-point device. Each N8 board is a self-contained test agent: it drives a DL MCU under test over UART, serves its own local web UI and REST API for on-bench control, and continuously reports live status to a central MQTT broker. A backend server subscribes to that broker, persists test history, and serves a fleet-wide web dashboard. The platform therefore consists of four logical tiers:

- N8 Dev Board (Test Controller) — runs the test firmware, hosts the local web server and REST API, and publishes test results via MQTT.
- MQTT Broker (your server) — receives live test data from every connected N8 board.
- Backend Server + Web Application (your server) — subscribes to MQTT topics, stores test history, and serves the live dashboard.
- DL MCU (Device Under Test) — the door-lock controller being tested, connected locally to a specific N8 board.

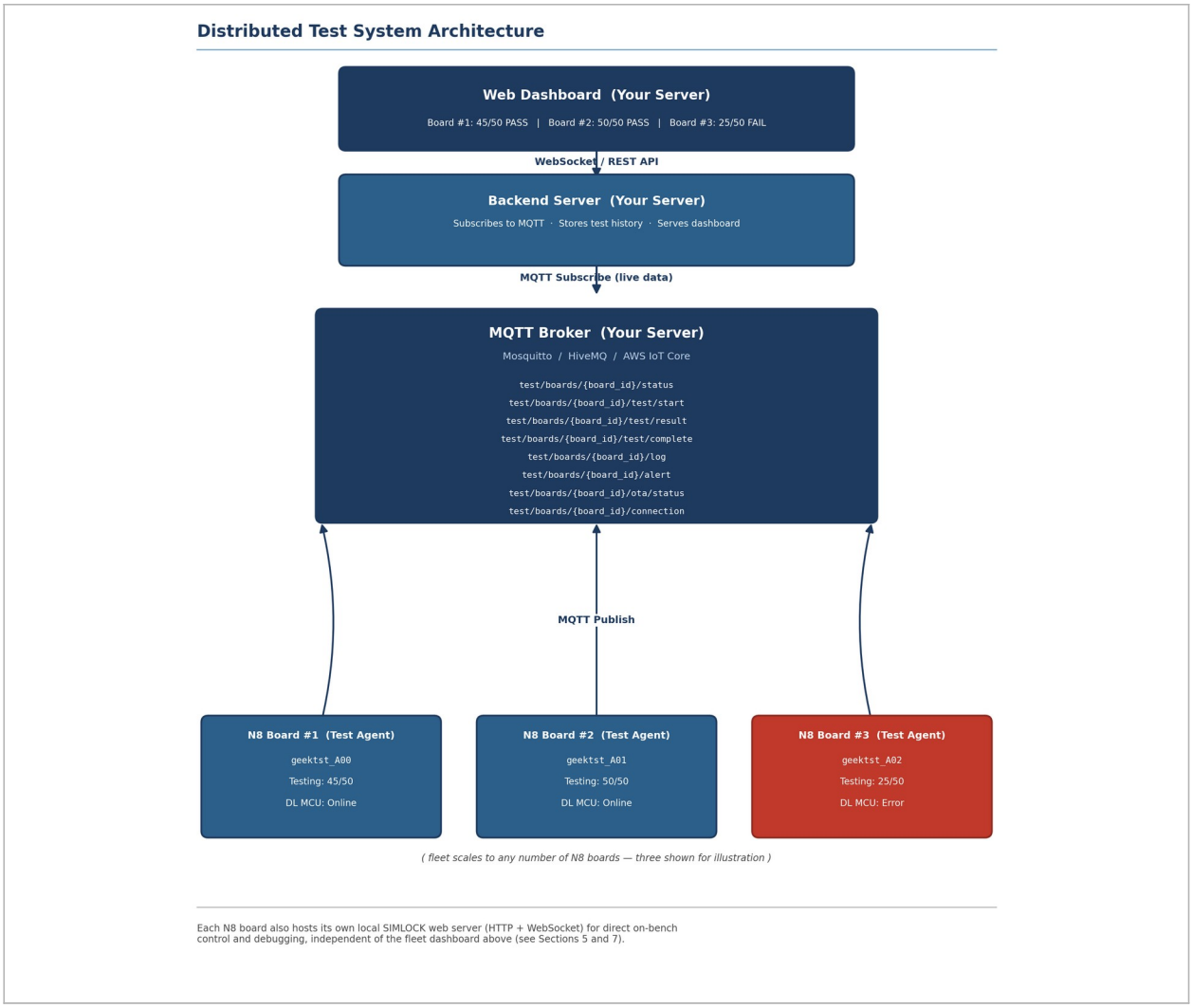

Figure 1 — Distributed test system architecture: N8 board fleet, MQTT broker, backend server, and dashboard.

2.2 Two Independent Interfaces

Each N8 board exposes two independent interfaces that serve different purposes and should not be confused:

- Local control (Section 7): a REST API and WebSocket, served directly by the N8 over Wi-Fi, for direct on-bench control and debugging of a single board — unchanged in spirit from Version 1.0.
- Fleet reporting (Section 8, new): an MQTT publish channel used by every N8 board to report heartbeat, test results, logs, alerts, and OTA status to the central broker, and to receive remote commands back from the backend.

2.3 Component Descriptions

Table 2 — System Components

Component	Description
N8 (ESP32-C6-DevKitC-1-N8)	Test controller. Hosts the local SIMLOCK web application and REST API; performs SoftAP provisioning; bridges commands to the DL MCU over UART; optionally interfaces a Camera MCU over SPI; publishes fleet telemetry over MQTT.
DL MCU (Door-Lock Controller)	Target device under test. Receives command frames from N8, drives lock hardware, manages enrolled credentials, and reports status back to N8.
Camera MCU	Optional image/vision module connected to N8 over SPI. Enabled by the N8's full GPIO breakout; not possible on the previous GEEK platform.
MQTT Broker (your server)	Central message broker (e.g., Mosquitto, HiveMQ, or AWS IoT Core) that receives publications from every N8 board and relays them to subscribed backend services (Section 8).
Backend Server (your server)	Subscribes to MQTT topics, stores test history in a database, exposes it to the dashboard over WebSocket/REST.
Web Dashboard (your server)	Fleet-wide monitoring UI: live board status, pass/fail counts, alerts, and OTA progress across all connected N8 boards.
SIMLOCK Web UI (per board)	Local, browser-based control and monitoring interface served directly by an individual N8; used for direct lock control, credential management, alert simulation, configuration, and log retrieval on that board.
Keypad / RFID Reader / Fingerprint Sensor / Buzzer / Backlight LED / Bolt Actuator	Physical DL MCU peripherals exercised by the functional requirements in Section 6.

3. Hardware Interface Specification

3.1 Why the N8 Dev Board

The N8 (ESP32-C6-DevKitC-1-N8) replaces the previous Waveshare GEEK stick as the test controller. The GEEK's limited GPIO breakout (GPIO3, 4, 5, 6, and 9 only) was sufficient for the DL MCU's 5-wire interface but left no pins free for a camera connector. The N8 exposes its full GPIO set, so both the DL MCU interface and a new Camera MCU (SPI) interface can be used at the same time.

Table 3 — GEEK vs. N8 Comparison

Aspect	GEEK	N8 Dev Board	Decision
IO pins available	Limited (GPIO3, 4, 5, 6, 9 only)	Full GPIO breakout (all pins)	N8 is better
Camera MCU connector	Not possible (insufficient pins)	Yes (SPI + GPIOs available)	N8 is better
DL MCU connector	Possible (5-wire uses GPIO3–6)	Yes (5-wire + spare pins)	Both work
Form factor	USB stick	Standard dev board	N8 for bench testing

3.2 N8 Dev Board Specifications

Table 4 — N8 Dev Board Specifications

Feature	Specification
Module	ESP32-C6-WROOM-1-N8 (8 MB flash)
USB	USB-C, for power, programming, and serial debug
IO breakout	All GPIOs available on header pins
Antenna	On-board PCB antenna
Wi-Fi Provisioning	SoftAP + captive portal (no companion app required)
Control Interface	Built-in web server (REST + WebSocket) plus MQTT reporting
Firmware Updates	OTA from a remote update server (Section 7.10)

3.3 N8 ↔ DL MCU and Camera MCU Pinout

Table 5 lists the DL MCU connector, unchanged from Version 1.0. Table 6 lists the new Camera MCU connector, made possible by the N8's full GPIO breakout. Figure 2 shows both connectors together.

Table 5 — N8 ↔ DL MCU Wiring (5-Wire)

Signal	N8 GPIO	DL MCU Pin	Direction
TXD	GPIO5	RX (MCU UART)	N8 → DL MCU
RXD	GPIO6	TX (MCU UART)	DL MCU → N8
SRDY	GPIO3	SRDY	DL MCU → N8 (active-low)
MRDY	GPIO4	MRDY	N8 → DL MCU (active-low)

Signal	N8 GPIO	DL MCU Pin	Direction
GND	GND	GND	Common ground

HARDWARE CAUTION

Per Espressif's ESP32-C6 documentation, GPIO4 and GPIO5 — used here for MRDY and TXD — are strapping pins, sampled at power-on/reset to help select the chip's boot mode. If the DL MCU actively drives these lines while N8 is powering on, it could interfere with a normal boot. Confirm with the DL MCU firmware team that MRDY/TXD are held in a safe, non-conflicting state (e.g., high-impedance) until N8 has finished booting, or add appropriate pull resistors / power-up sequencing. GPIO3, GPIO6, and the Camera MCU's GPIO18–21 (Table 6) are not strapping pins and do not carry this concern.

Table 6 — N8 ↔ Camera MCU Wiring (SPI)

Signal	N8 GPIO	Camera MCU Pin	Direction
SCK	GPIO19	SCK	N8 → Camera
MOSI	GPIO18	MOSI	N8 → Camera
MISO	GPIO20	MISO	Camera → N8
CS	GPIO21	CS	N8 → Camera
3.3V	3.3V	VCC	Power
GND	GND	GND	Common ground

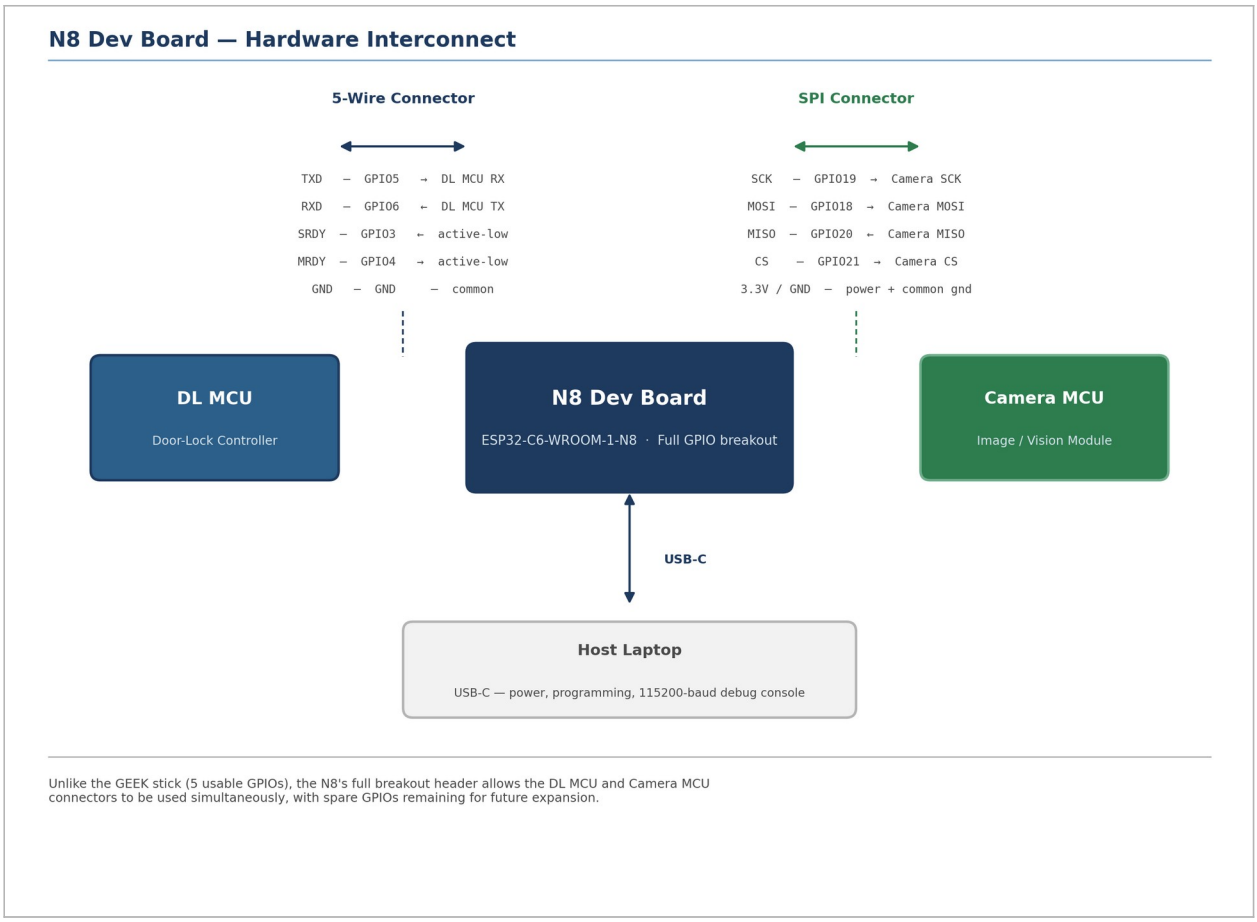

Figure 2 — N8 hardware interconnect: DL MCU 5-wire connector and Camera MCU SPI connector.

3.4 Host Connection — Power, Programming, and Debug

N8 connects to a laptop host via USB-C, which supplies power and provides the programming/debug interface. A serial monitor opened on the host at 115200 baud displays N8 boot messages and firmware debug/trace output. After initial programming, N8 operates standalone over Wi-Fi — the USB connection is not required for normal operation, including MQTT fleet reporting.

4. Communication Protocol

4.1 UART Configuration

The USB debug/serial-monitor link between N8 and the host laptop operates at 115200 baud (Section 3.4). The baud rate of the physical 5-wire UART link between N8 and the DL MCU was not stated in the source material and should be confirmed against the DL MCU firmware/protocol reference before it is assumed to match.

4.2 Handshake Protocol (MRDY / SRDY)

Every UART transaction between N8 and the DL MCU is gated by a two-wire, active-low handshake, shown in Figure 3. N8 asserts MRDY (Master Ready) low to signal that it has a frame to send. The DL MCU responds by asserting SRDY (Slave Ready) low once it is prepared to receive. Only after this handshake completes does N8 transmit the command frame on TXD; the DL MCU replies on its own TX line, after which both handshake lines are released (returned high) and the bus returns to idle. This handshake is unchanged from Version 1.0 — the underlying GPIO assignments (3–6) carried over directly to the N8.

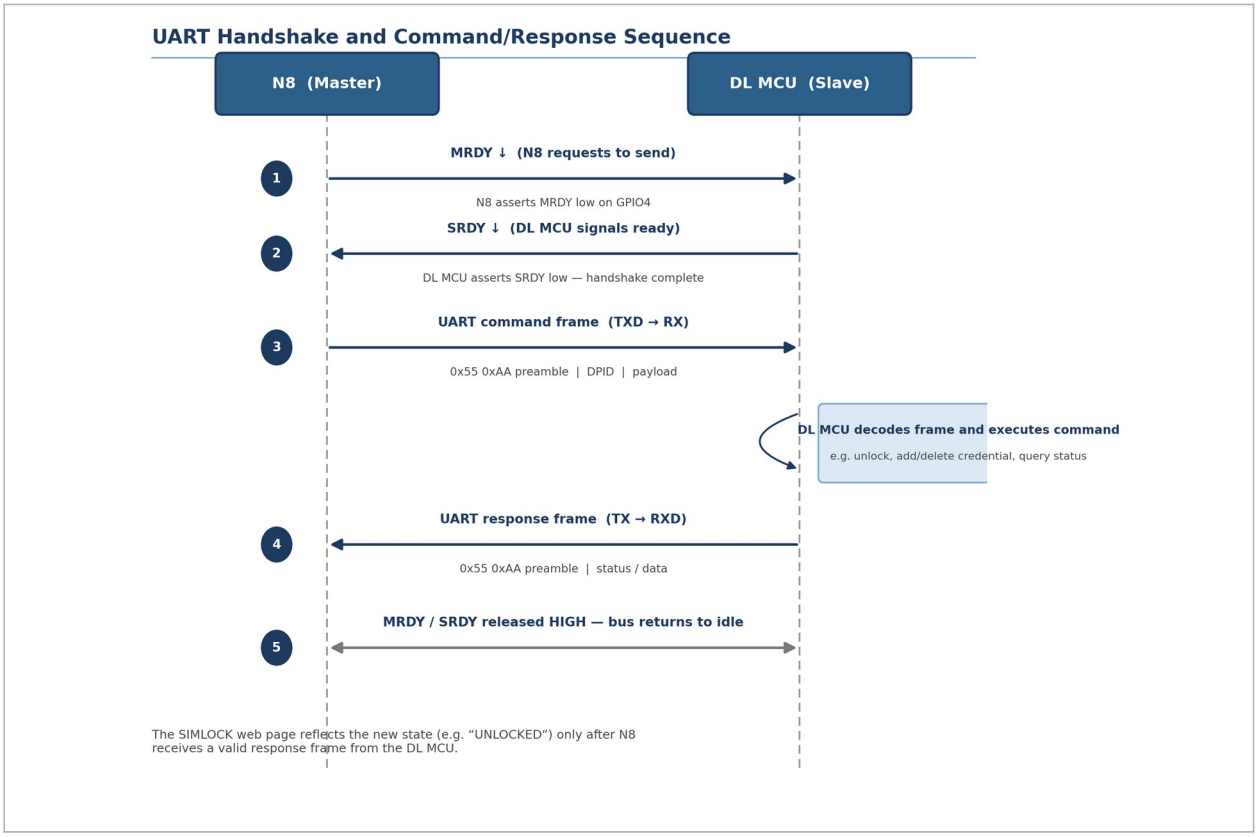

Figure 3 — UART handshake and command/response sequence between N8 and the DL MCU.

4.3 Frame Format

Command and response frames observed in the source material begin with a two-byte preamble, 0x55 0xAA, followed by a Data Point Identifier (DPID) that selects the operation and a variable-length payload.

4.4 DPID Command Reference (Reserved)

This revision was requested to document a complete 41-command DPID set. Only two DPID values are actually confirmed in the available source material; the rest are not enumerated. Table 7 records what is known and explicitly reserves the remainder rather than inventing numeric codes for a physical access-control protocol.

Table 7 — DPID Command Reference

DPID	Operation	Status
21	Add Credential (PIN / RFID / Fingerprint) — supports one-time and duration flags	Confirmed (Tests 5, 6, 7, 13, 18)
22	Delete Credential by Hardware ID	Confirmed (Tests 8, 14, 19)
Not yet assigned	Remaining command set (~39 additional operations across lock control, status/read queries, buzzer, backlight, alerts, configuration, system management, and OTA triggering — see Section 6)	Reserved / TBD

RESERVED / TO BE DEFINED

Only DPID 21 and 22 are confirmed. The remaining ~39 operations are enumerated by name throughout Section 6, but no numeric DPID has been assigned to them in any source material provided to date. Completing this table requires the DL MCU firmware team's protocol reference — it is not something this specification can responsibly infer.

4.5 Credential Data Structure (Reserved)

The exact byte-level layout of a credential record has not been supplied. The logical attributes below are inferred from the functional requirements in Sections 6.2–6.4, which describe what a credential must carry, without describing how it is encoded on the wire.

Table 8 — Known Logical Credential Attributes

Attribute	Description	Source
Hardware ID	Unique identifier assigned to the credential upon creation; used for deletion and listing.	Tests 8, 10, 14, 16, 19, 21
Credential Type	PIN, RFID, or Fingerprint.	Sections 6.2–6.4
Value / Template Reference	The PIN digits, RFID card ID, or fingerprint template/signature reference.	Tests 5, 12, 17
One-Time Flag	Marks a PIN as single-use (OTP); rejected after first successful use.	Test 5
Expiry / Duration	Timestamp or duration after which the credential is rejected; omitted entirely for permanent credentials.	Tests 6, 7, 13, 18

RESERVED / TO BE DEFINED

The binary encoding of a credential record — byte offsets, field widths, endianness — as transmitted in a DPID 21 payload was not supplied and is Reserved / TBD pending the DL MCU protocol reference.

4.6 Error Handling (Reserved)

The source material implies some error-adjacent behavior indirectly, without defining a formal scheme:

- A command with no valid response frame leaves the web UI unchanged rather than transitioning state (Section 4.2).
- Repeated incorrect PIN entries trigger a configurable lockout after a set number of attempts (Section 6.9, Test 39).
- The MQTT OTA status message includes an error_message field (Section 8.4), though its possible values are not enumerated.

RESERVED / TO BE DEFINED

No formal error-code table, HTTP status-code convention, or standardized error-response schema was supplied for the REST API, MQTT payloads, or UART responses. This is Reserved / TBD pending confirmation from the firmware team.

ILLUSTRATIVE EXAMPLE — NOT CONFIRMED

The structure below illustrates the shape such a table could take once the firmware team defines real values. None of these codes are confirmed — do not implement against them.

Table 9 — Illustrative Error Code Structure (Example Only)

Code	Meaning (example)
0x00	OK / Success
0x01	Invalid Command / Unknown DPID
0x02	Invalid Parameter / Malformed Payload
0x03	Credential Not Found
0x04	Credential Storage Full
0x05	Handshake Timeout (no SRDY response)
0x06	DL MCU Busy
0xFF	Unspecified Error

NOT CONFIRMED

Reminder: illustrative only. Confirm real codes and their meanings with the DL MCU firmware team before use.

5. Device Provisioning

Provisioning was simplified in Version 2.0: the BLE + Web Bluetooth path used by GEEK has been removed. N8 provisions exclusively via SoftAP and a captive portal, then immediately connects to the MQTT broker as its final setup step.

5.1 First Boot (No Wi-Fi Credentials)

When an N8 board boots with no saved Wi-Fi credentials, it follows the sequence in Table 10.

Table 10 — First-Boot Provisioning Sequence

Step	Action
1	N8 starts in SoftAP mode (SSID: ESP32-Setup-XXXX).
2	Laptop/phone connects to this SSID (no password required).
3	Browser automatically opens to the captive portal (http://192.168.4.1).
4	User selects a Wi-Fi network and enters its password.
5	N8 saves the credentials to NVS.
6	N8 switches to STA mode and connects to Wi-Fi.
7	N8 displays its IP address (on the serial monitor or LCD).
8	User navigates to http://<N8_IP>/ for the local SIMLOCK web interface.
9	N8 connects to the MQTT broker and starts publishing status.

IMPLEMENTATION GUIDANCE

This specification assumes the MQTT broker address (Table 33, e.g., mqtt.ebizco.com.au) is preconfigured — either compiled into firmware or stored in NVS alongside the Wi-Fi credentials. For a fixed fleet pointed at a single broker, this is normally sufficient. A provisioning-time override (e.g., a broker-address field added to the captive portal in step 4) is out of scope for this revision, but could be added later if the broker address needs to vary per deployment.

5.2 Provisioning Flow

Figure 4 consolidates the decision logic and the single SoftAP provisioning path, including the new MQTT connection step at the end.

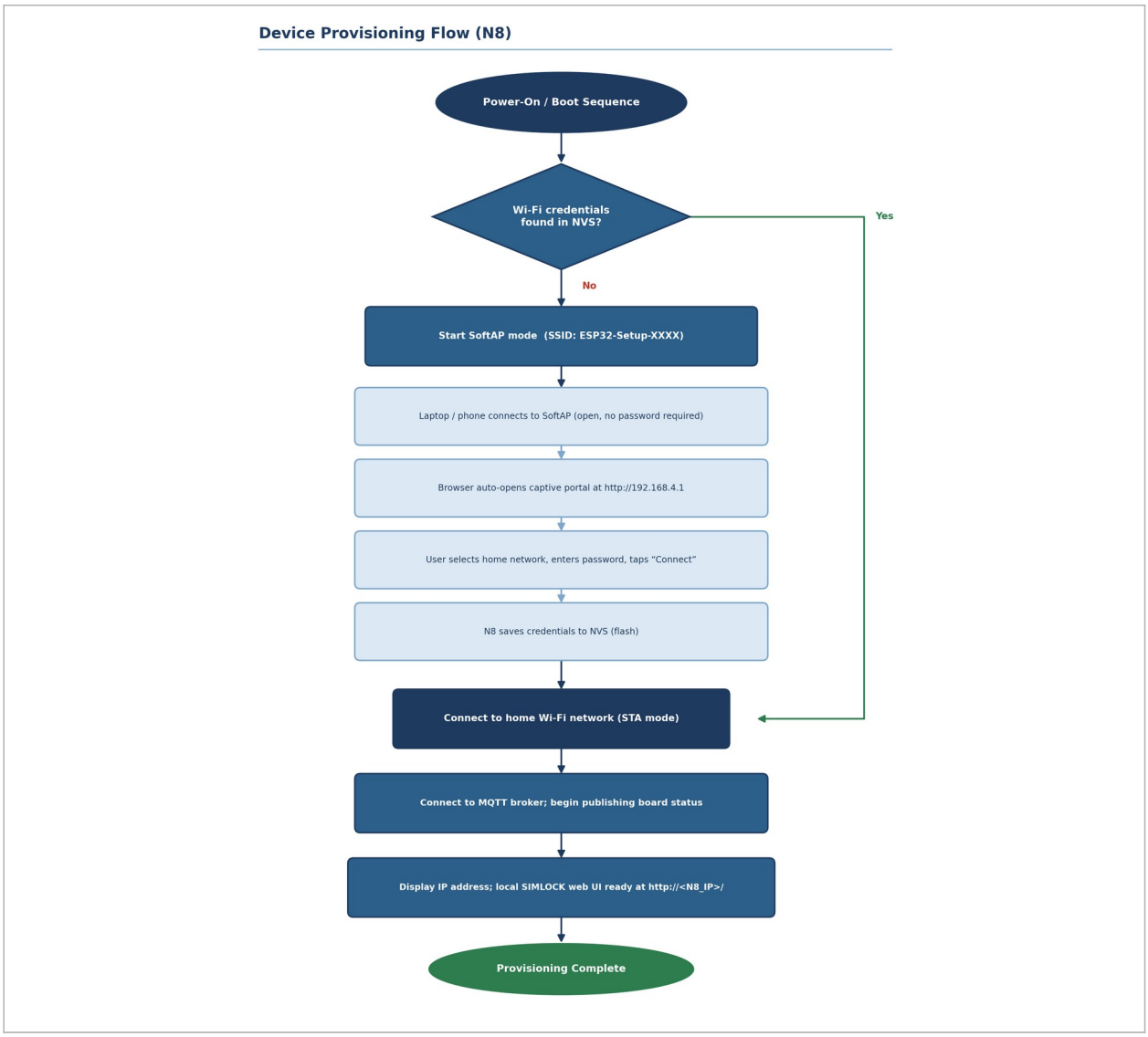

Figure 4 — N8 provisioning flow: SoftAP + captive portal, followed by MQTT broker connection.

5.3 Post-Provisioning Verification

Table 11 — Post-Provisioning Verification Steps

Step	Action	Expected Result
1	Note the IP address shown on the serial monitor.	—
2	Open a browser to http://<N8_IP>/.	The local SIMLOCK web page loads.
3	Check the MQTT connection.	Board status appears in the fleet dashboard.
4	Record the firmware version.	geektst_A00 (or current version) is displayed.

6. Functional Specification

This section specifies the required behavior of each SIMLOCK feature area, organized to mirror the ten functional groupings used in the source test procedure. Each requirement is written in “shall” form and cross-referenced to the originating test case(s) (see also Section 9, Requirements Traceability Matrix).

6.1 Direct Lock Control

Table 12 — Direct Lock Control Requirements

Function	Requirement	Ref.
Open Door	The system shall transmit an unlock command frame to the DL MCU following the MRDY/SRDY handshake (Section 4.2). Upon a valid response, the web UI shall update to the “UNLOCKED” state.	Test 1
Lock Door	The system shall transmit a lock command frame to the DL MCU. Upon a valid response, the web UI shall update to the “LOCKED” state.	Test 2
Get Door Status	The system shall query and report the physical door open/closed state on request.	Test 3
Get Lock Status	The system shall query and report the current locked/unlocked state on request.	Test 4

6.2 PIN Credential Management

Table 13 — PIN Credential Management Requirements

Function	Requirement	Ref.
Set One-Time PIN (OTP)	The system shall add a PIN credential flagged as single-use (DPID 21). The PIN shall unlock the door on first use and shall be rejected on any subsequent attempt.	Test 5
Set PIN with Duration	The system shall add a PIN credential with a configurable expiry duration (default 10 minutes). The PIN shall be accepted until the duration elapses, after which it shall be rejected.	Test 6
Set Permanent Password	The system shall add a PIN credential with no expiry. The credential shall remain valid indefinitely.	Test 7
Delete PIN by Hardware ID	The system shall remove a specific PIN credential identified by its Hardware ID (DPID 22). The credential shall be rejected on subsequent use.	Test 8
Delete All PINs	The system shall remove all enrolled PIN credentials on a confirmed bulk-delete request.	Test 9
List All PINs	The system shall return, for every enrolled PIN, its Hardware ID, value, and expiry.	Test 10
Unlock with PIN (web)	The system shall accept a PIN submitted through the web UI, forward a verification request to the DL MCU, and unlock the door on a successful match.	Test 11

6.3 RFID Credential Management

Table 14 — RFID Credential Management Requirements

Function	Requirement	Ref.
Read RFID Card	The system shall read and report the identifier of an RFID card presented to the DL MCU reader.	Test 12
Add RFID with Duration	The system shall enroll an RFID card with a configurable expiry duration (default 10 minutes). The card shall be accepted until expiry, after which it shall be rejected.	Test 13
Delete RFID by Hardware ID	The system shall remove a specific RFID credential by Hardware ID. The card shall be rejected on subsequent use.	Test 14
Delete All RFID Cards	The system shall remove all enrolled RFID credentials on a confirmed bulk-delete request.	Test 15
List All RFID Cards	The system shall return, for every enrolled card, its Hardware ID, card ID, and expiry.	Test 16

6.4 Fingerprint Credential Management

Table 15 — Fingerprint Credential Management Requirements

Function	Requirement	Ref.
Read Fingerprint	The system shall capture and confirm a fingerprint signature presented to the DL MCU sensor.	Test 17
Add Fingerprint with Duration	The system shall enroll a fingerprint with a configurable expiry duration (default 5 minutes). The fingerprint shall be accepted until expiry, after which it shall be rejected.	Test 18
Delete Fingerprint by Hardware ID	The system shall remove a specific fingerprint credential by Hardware ID. It shall be rejected on subsequent use.	Test 19
Delete All Fingerprints	The system shall remove all enrolled fingerprint credentials on a confirmed bulk-delete request.	Test 20
List All Fingerprints	The system shall return, for every enrolled fingerprint, its Hardware ID and expiry.	Test 21

6.5 Credential Lifecycle Model

PIN, RFID, and fingerprint credentials share a common lifecycle, shown in Figure 5. A credential enters the Active state when added via DPID 21. From there it follows exactly one of three terminal paths, depending on how it was configured: a one-time credential becomes Consumed after its first successful use; a duration-limited credential becomes Expired once its configured time elapses; and any credential — including a permanent one — becomes Deleted on an explicit delete command (DPID 22) or a factory reset. A permanent credential (no expiry configured) remains Active indefinitely until explicitly deleted.

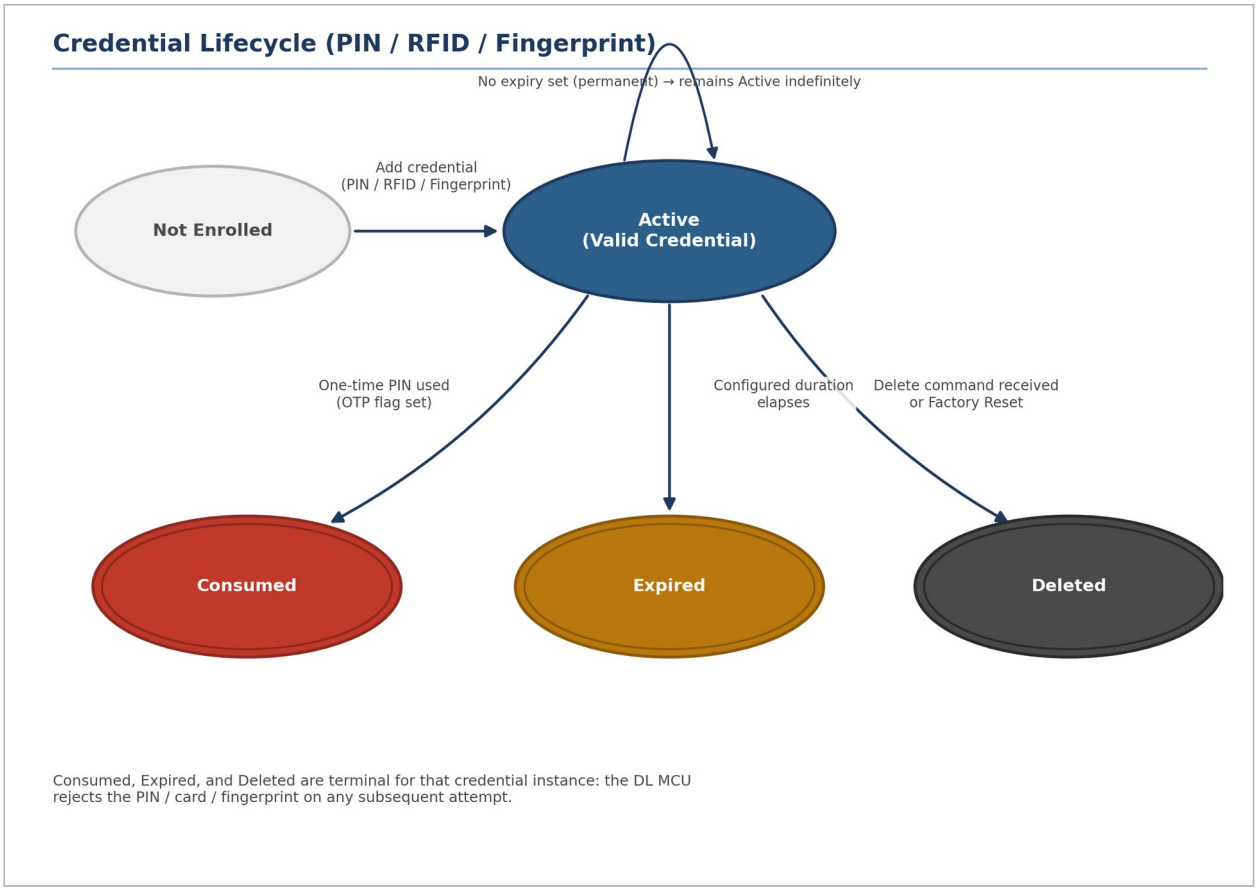

Figure 5 — Credential lifecycle shared by PIN, RFID, and fingerprint credentials.

6.6 Alerts & Security

Table 16 — Alerts & Security Requirements

Function	Requirement	Ref.
Buzzer: Single Beep	The system shall produce one short audible beep on request.	Test 22
Buzzer: Double Beep	The system shall produce two short audible beeps on request.	Test 23
Buzzer: Alarm Pattern	The system shall produce a three-beep / one-second-silence / three-beep pattern on request.	Test 24
Failed-Attempt Alert (Fingerprint)	The system shall log an alert categorized “Fingerprint failed attempts” after repeated unregistered-fingerprint presentations.	Test 25
Stuck Bolt Alert	The system shall log an alert categorized “Stuck Bolt” when the condition is triggered.	Test 26
High Temperature Alert	The system shall log an alert categorized “High Temperature” when the condition is triggered.	Test 27
Duress Alarm	The system shall log a “Duress Alarm” event while producing no audible output (silent alarm).	Test 28

6.7 Backlight Control

Table 17 — Backlight Control Requirements

Function	Requirement	Ref.
Flash 2 Times	The keypad backlight shall flash twice on request.	Test 29
Backlight ON	The keypad backlight shall turn on and remain on until commanded otherwise.	Test 30
Backlight OFF	The keypad backlight shall turn off on request.	Test 31

6.8 Logs & Status Reporting

Table 18 — Logs & Status Reporting Requirements

Function	Requirement	Ref.
Transaction Log	The system shall record and return unlock events, including the credential method used (PIN/RFID/Fingerprint) and a timestamp.	Test 32
Alert History	The system shall record and return triggered alert events.	Test 33
Battery Level	The system shall report the current battery level as a percentage.	Test 34
Wi-Fi Signal Strength (RSSI)	The system shall report the current Wi-Fi RSSI in dBm.	Test 35
Firmware Version	The system shall report the current firmware version string (format geektst_<letter><2-digit>, e.g., geektst_A00).	Test 36

6.9 Configuration Parameters

Table 19 — Configuration Requirements

Function	Requirement	Ref.
Auto-Lock Timer	The system shall re-lock the door automatically after a configurable delay (e.g., 5 s / 10 s / 30 s) following an unlock.	Test 37
Auto-Lock Enable/Disable	The system shall support enabling or disabling automatic re-locking. When disabled, the door shall remain unlocked until a manual lock command is issued.	Test 38
Wrong-Attempt Lockout	The system shall lock out keypad entry for a configurable duration (e.g., 60 s) after a configurable number of consecutive incorrect PIN attempts (e.g., 5). Correct PINs shall be rejected during lockout and accepted once it expires.	Test 39
Scramble PIN	The system shall support an optional “scramble PIN” mode in which junk digits surrounding the true PIN are still accepted. When disabled, only the exact PIN shall be accepted.	Test 40
Volume Level	The system shall support configurable buzzer volume levels (e.g., low / medium / high).	Test 41
Language	The system shall support a configurable UI/keypad language (e.g., Vietnamese).	Test 42

Function	Requirement	Ref.
Hardware ID List	The system shall return a consolidated list of Hardware IDs for all enrolled credential types (PIN, RFID, fingerprint).	Test 43

6.10 System Management

Table 20 — System Management Requirements

Function	Requirement	Ref.
Factory Reset	The system shall delete all enrolled credentials and clear all logs on a confirmed factory-reset request.	Test 44
Reboot Device	The system shall support a remote reboot request that resets N8.	Test 45
Device Info	The system shall report manufacturer, model, and serial number on request.	Test 46
Export Logs	The system shall support exporting logs as a downloadable file (CSV or JSON).	Test 47

6.11 Firmware Version Cycling & OTA Update

Table 21 — Version Cycling & OTA Requirements

Function	Requirement	Ref.
Version Cycling	Triple-clicking the boot button (three presses within three seconds) shall cause N8 to reset and increment its firmware version identifier (format geektst_<letter><2-digit>, e.g., geektst_A00 → A01, continuing through A99 → B00, through Z99 → A00). The new version shall persist across reboots via NVS. See Section 7.10 for the full versioning scheme.	Test 48
OTA Update Check	After a triple-click reset (or a remote MQTT “check_ota” command, Section 8.5), N8 shall connect to Wi-Fi, fetch version.json from the OTA server (Section 7.10), compare firmware versions, and report “No update available” when the server version matches the current version.	Test 49
Triple-Click Boot Detection	N8 shall detect three boot-button presses within a three-second window, set a “force_update” flag in NVS, and reset. On the subsequent boot it shall enter its update-check routine and clear the flag.	Test 50

7. REST API Reference

Each N8 board exposes the following REST and WebSocket endpoints locally, for the on-bench SIMLOCK web client described in Section 2.2. These are distinct from the MQTT fleet-reporting channel described in Section 8. All request/response payload schemas beyond the endpoint and method named here were not specified in the source material.

7.1 Lock Control

Table 22 — Lock Control Endpoints

Endpoint	Method	Description
/api/lock/unlock	POST	Unlock door
/api/lock/lock	POST	Lock door
/api/lock/status	GET	Get door and lock status

7.2 PIN Credential Management

Table 23 — PIN Credential Management Endpoints

Endpoint	Method	Description
/api/credential/pin/otp	POST	Set One-Time PIN
/api/credential/pin/duration	POST	Set PIN with expiry duration
/api/credential/pin/permanent	POST	Set permanent password
/api/credential/pin/{id}	DELETE	Delete PIN by Hardware ID
/api/credential/pin/all	DELETE	Delete all PINs
/api/credential/pin/list	GET	List all PINs
/api/credential/pin/verify	POST	Unlock with PIN

7.3 RFID Credential Management

Table 24 — RFID Credential Management Endpoints

Endpoint	Method	Description
/api/credential/rfid/read	POST	Read RFID card
/api/credential/rfid/duration	POST	Add RFID with expiry
/api/credential/rfid/{id}	DELETE	Delete RFID by Hardware ID
/api/credential/rfid/all	DELETE	Delete all RFID cards
/api/credential/rfid/list	GET	List all RFID cards

7.4 Fingerprint Credential Management

Table 25 — Fingerprint Credential Management Endpoints

Endpoint	Method	Description
/api/credential/finger/read	POST	Read fingerprint
/api/credential/finger/duration	POST	Add fingerprint with expiry
/api/credential/finger/{id}	DELETE	Delete fingerprint by Hardware ID
/api/credential/finger/all	DELETE	Delete all fingerprints
/api/credential/finger/list	GET	List all fingerprints

7.5 Alerts & Buzzer

Table 26 — Alerts & Buzzer Endpoints

Endpoint	Method	Description
/api/buzzer/beep	POST	Single beep
/api/buzzer/double	POST	Double beep
/api/buzzer/alarm	POST	Alarm pattern
/api/alert/failed-attempt	POST	Trigger failed-attempt alert
/api/alert/stuck-bolt	POST	Trigger stuck-bolt alert
/api/alert/high-temp	POST	Trigger high-temperature alert
/api/alert/duress	POST	Trigger duress alarm

7.6 Backlight Control

Table 27 — Backlight Control Endpoints

Endpoint	Method	Description
/api/backlight/flash	POST	Flash backlight 2 times
/api/backlight/on	POST	Turn backlight ON
/api/backlight/off	POST	Turn backlight OFF

7.7 Logs & Status

Table 28 — Logs & Status Endpoints

Endpoint	Method	Description
/api/log/transaction	GET	Get transaction log
/api/log/alert	GET	Get alert history
/api/status/battery	GET	Get battery level

Endpoint	Method	Description
/api/status/rssi	GET	Get Wi-Fi RSSI
/api/version	GET	Get firmware version

7.8 Configuration

Table 29 — Configuration Endpoints

Endpoint	Method	Description
/api/config/autolock	POST	Set Auto-Lock timer
/api/config/autolock/enable	PUT	Enable/Disable Auto-Lock
/api/config/lockout	POST	Set wrong-attempt lockout
/api/config/scramble	PUT	Enable/Disable Scramble PIN
/api/config/volume	PUT	Set volume level
/api/config/language	PUT	Set language
/api/credential/list/all	GET	Get Hardware ID list

7.9 System Management

Table 30 — System Management Endpoints

Endpoint	Method	Description
/api/system/factory-reset	POST	Factory reset
/api/system/reboot	POST	Reboot device
/api/system/info	GET	Get device info
/api/log/export	GET	Export logs

7.10 OTA & Version Management

Table 31 — OTA & Version Management Endpoints

Endpoint	Method	Description
/api/version	GET	Get current firmware version
/api/ota/check	POST	Check for OTA update
/api/ota/status	GET	Get OTA update status
/api/ota/update	POST	Download and install firmware update
/api/ota/rollback	POST	Rollback to previous firmware version
/api/ota/trigger	POST	Trigger OTA via triple-click simulation
/api/ota/config	GET / PUT	Get / Set OTA configuration
/api/ota/history	GET	Get OTA update history

OTA Firmware Versioning Scheme

Version format: geektst_ followed by a letter (A–Z) and a two-digit number (00–99).

Version progression: A00 → A01 → A02 → … → A99, then B00 → B01 → … → B99, then C00 → … → Z99. After Z99, the sequence wraps to A00.

Firmware binary naming: geektst_A00.ino, geektst_A01.ino, …, geektst_A99.ino, geektst_B00.ino, and so on — one .ino file per version.

OTA server URL: https://ebizco.com.au/doorlock/test/dle32/

Version manifest URL: https://ebizco.com.au/doorlock/test/dle32/version.json

N8 compares its current version against the latest_version field of the manifest below to decide whether an update is available:

```
{
  "latest_version": "geektst_A02",
  "release_date": "2026-08-04",
  "changelog": "Fixed Wi-Fi reconnection bug, improved handshake reliability",
  "size_bytes": 1048576,
  "md5": "a3b7c9d1e5f2a4b6c8d0e1f2a3b4c5d6",
  "url": "https://ebizco.com.au/doorlock/test/dle32/geektst_A02.ino",
  "force_update": false
}
```

IMPLEMENTATION GUIDANCE

The md5 field above provides basic integrity checking, which is generally adequate for a controlled test environment. For production deployments, consider adding a cryptographic firmware signature (signed with a private key and verified on-device) in addition to, or instead of, the MD5 checksum — MD5 alone does not protect against a deliberately modified binary.

7.11 WebSocket Real-Time Updates

Table 32 — WebSocket Endpoint

Endpoint	Method	Description
/ws	WebSocket	Real-time status updates

The WebSocket connection mirrors REST/local state (lock status, credential events, alerts) to the local SIMLOCK web UI in real time, distinct from the MQTT fleet channel in Section 8.

RESERVED / TO BE DEFINED

Only the /ws endpoint path is confirmed. The message payload schema — event types, JSON field names, and framing — was not supplied and is Reserved / TBD pending confirmation from the firmware team.

ILLUSTRATIVE EXAMPLE — NOT CONFIRMED

The shapes below illustrate what event-style WebSocket messages might look like. Field names and event types are guesses for illustration — not confirmed values.

```
{
  "event": "lock_state",
  "value": "locked",
  "timestamp": "2026-08-04T10:30:00Z"
}
```

```
{
  "event": "credential_added",
  "credential_type": "pin",
  "hardware_id": "HW-0007"
}
```

NOT CONFIRMED

Reminder: illustrative only. Confirm the real event/field names with the firmware team before use.

8. MQTT Reporting Specification

8.1 Overview

Each N8 test board publishes live test results to a central MQTT broker. The backend web application subscribes to these topics to display a live dashboard of all connected test boards (Section 2). This is new in Version 2.0 and is independent of the local REST/WebSocket API in Section 7.

8.2 MQTT Broker Configuration

Table 33 — MQTT Broker Configuration

Parameter	Value
Broker	mqtt.ebizco.com.au (or your server)
Port	1883 (non-TLS) or 8883 (TLS)
Client ID	N8-{MAC} or N8-{serial}
QoS	1 (at least once delivery)
Retain	Status topics retained

IMPLEMENTATION GUIDANCE

board_id (the {board_id} placeholder used throughout every topic in Table 34) is not defined at the protocol level in the source material. A practical approach is to derive it deterministically from the N8's MAC address (e.g., N8-{last 6 hex digits of MAC}) or have the backend assign it during first connection. Whichever approach is used, board_id must be unique within the fleet, since it doubles as the MQTT Client ID above.

IMPLEMENTATION GUIDANCE

Table 33 lists port 8883 for TLS, but the source material does not mandate its use. For production or sensitive test deployments, connect over TLS (port 8883) with a server certificate, and configure broker-side username/password authentication if required, with credentials stored in NVS rather than compiled into firmware.

8.3 MQTT Topic Structure

Table 34 — MQTT Topics

Topic	Payload	Frequency	Description
test/boards/{board_id}/status	JSON	Every 10 seconds	Board health: Wi-Fi RSSI, uptime, firmware version
test/boards/{board_id}/test/start	JSON	On test start	Test suite started, test count
test/boards/{board_id}/test/result	JSON	On each test completion	Individual test result (PASS/FAIL)
test/boards/{board_id}/test/complete	JSON	On test suite complete	Summary: total passed, failed, skipped
test/boards/{board_id}/log	JSON	On each log event	Detailed log messages

Topic	Payload	Frequency	Description
test/boards/{board_id}/alert	JSON	On alert triggered	Alerts (failed attempts, stuck bolt, etc.)
test/boards/{board_id}/ota/ status	JSON	On OTA progress	OTA update progress
test/boards/{board_id}/ connection	JSON	On connect/disconnect	Board online/offline status

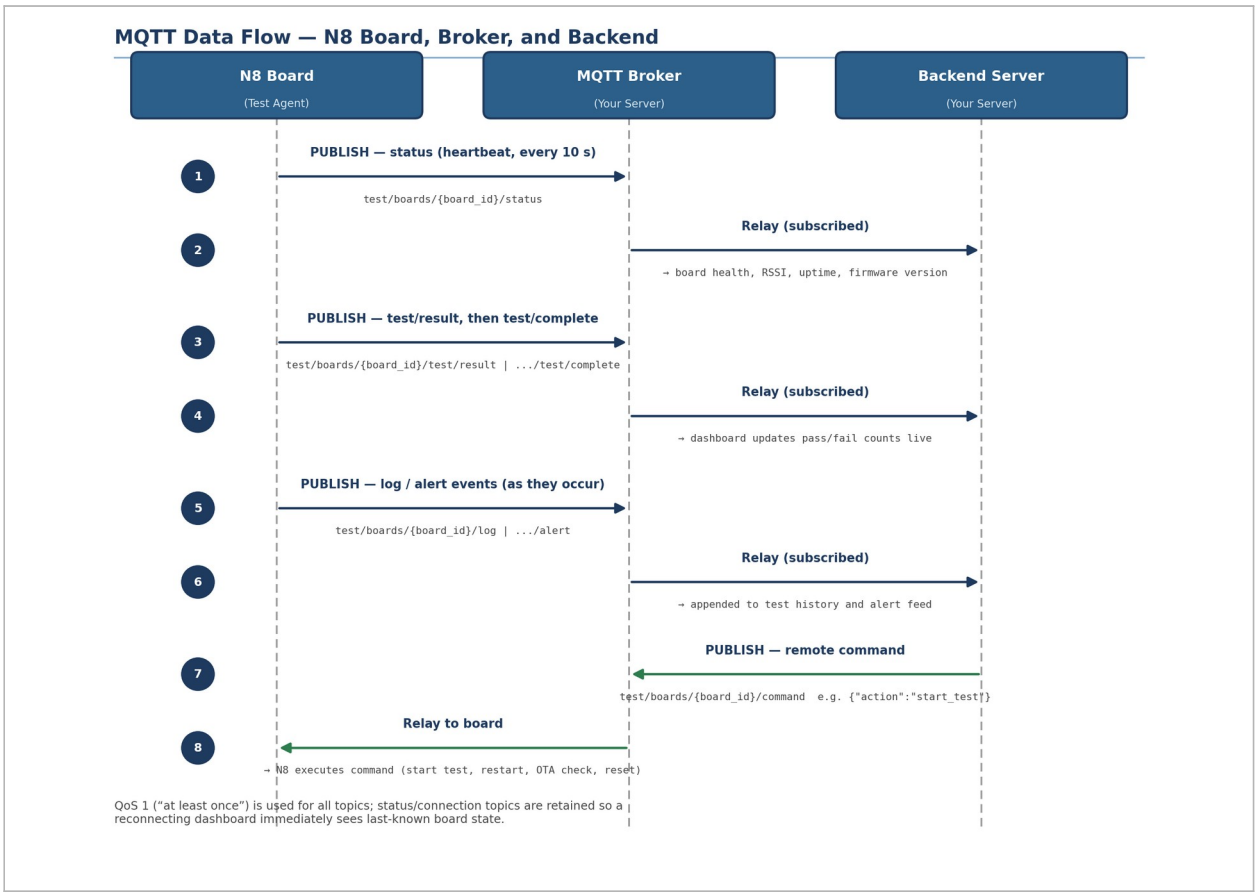

Figure 6 — MQTT data flow: N8 board publishes to the broker, which relays to the backend; remote commands flow in reverse.

8.4 MQTT Message Formats

Board Status (Heartbeat)

Topic: `test/boards/{board_id}/status`

```
{
  "board_id": "N8-001",
  "timestamp": "2026-08-04T10:30:00Z",
  "firmware_version": "geektst_A00",
  "uptime_seconds": 3600,
  "wifi": {
    "ssid": "Office-WiFi",
    "rssi": -55,
    "ip": "192.168.1.100"
  },
  "dl_mcu": {
    "connected": true,
    "last_heartbeat": "2026-08-04T10:29:55Z"
  },
  "heap_usage_percent": 42
}
```

Individual Test Result

Topic: `test/boards/{board_id}/test/result`

```
{
  "board_id": "N8-001",
  "timestamp": "2026-08-04T10:30:05Z",
  "test_id": 1,
  "test_name": "Open Door (Direct Unlock)",
  "section": "Direct Control",
  "status": "PASS",
  "duration_ms": 250,
  "details": "Door unlocked successfully"
}
```

Test Complete (Summary)

Topic: `test/boards/{board_id}/test/complete`

```
{
  "board_id": "N8-001",
  "timestamp": "2026-08-04T10:35:00Z",
  "summary": {
    "total": 50,
    "passed": 48,
    "failed": 2,
    "skipped": 0,
    "pass_rate": 96.0
  },
```

```
  "failed_tests": [
    {
      "test_id": 25,
      "name": "Failed-Attempt Alert",
      "error": "Alert not triggered"
    }
  ],
  "duration_seconds": 300
}
```

Log Message

Topic: `test/boards/{board_id}/log`

```
{
  "board_id": "N8-001",
  "timestamp": "2026-08-04T10:30:10Z",
  "level": "INFO",
  "source": "UART",
  "message": "PIN 123456 added with expiry 5 minutes"
}
```

Alert

Topic: `test/boards/{board_id}/alert`

```
{
  "board_id": "N8-001",
  "timestamp": "2026-08-04T10:30:15Z",
  "alert_type": "failed_attempt",
  "severity": "WARNING",
  "details": "Fingerprint failed attempts (5 attempts)"
}
```

OTA Status

Topic: `test/boards/{board_id}/ota/status`

```
{
  "board_id": "N8-001",
  "timestamp": "2026-08-04T10:30:20Z",
  "status": "downloading",
  "current_version": "geektst_A00",
  "target_version": "geektst_A02",
  "progress_percent": 45,
  "error_message": null
}
```

Board Connection Status

Topic: `test/boards/{board_id}/connection`

```
{
  "board_id": "N8-001",
  "timestamp": "2026-08-04T10:30:00Z",
  "status": "online"
}
```

8.5 Remote Commands (Server → Board)

The server can send commands to a board via MQTT:

Table 35 — Remote Commands

Command	Topic	Payload	Description
Start Test	test/boards/{board_id}/command	{"action": "start_test"}	Start full test suite
Restart Board	test/boards/{board_id}/command	{"action": "restart"}	Reboot board
Check OTA	test/boards/{board_id}/command	{"action": "check_ota"}	Check for OTA update
Factory Reset	test/boards/{board_id}/command	{"action": "factory_reset"}	Reset all credentials

9. Requirements Traceability Matrix

Table 36 maps every source test case to the specification section that formalizes it, supporting verification planning and change-impact analysis. Section 5 (Device Provisioning) and Section 8 (MQTT Reporting) are not part of the original 50-case test procedure and so are not numbered as Tests; they are covered narratively in this specification instead.

Table 36 — Test-to-Section Traceability

Test	Description	Section
1	Open Door	§6.1
2	Lock Door	§6.1
3	Get Door Status	§6.1
4	Get Lock Status	§6.1
5	Set One-Time PIN	§6.2
6	Set PIN with Duration	§6.2
7	Set Permanent Password	§6.2
8	Delete PIN by Hardware ID	§6.2
9	Delete All PINs	§6.2
10	List All PINs	§6.2
11	Unlock with PIN (web)	§6.2
12	Read RFID Card	§6.3
13	Add RFID with Duration	§6.3
14	Delete RFID by Hardware ID	§6.3
15	Delete All RFID Cards	§6.3
16	List All RFID Cards	§6.3
17	Read Fingerprint	§6.4
18	Add Fingerprint with Duration	§6.4
19	Delete Fingerprint by Hardware ID	§6.4
20	Delete All Fingerprints	§6.4
21	List All Fingerprints	§6.4
22	Buzzer: Single Beep	§6.6
23	Buzzer: Double Beep	§6.6
24	Buzzer: Alarm Pattern	§6.6
25	Failed-Attempt Alert	§6.6
26	Stuck Bolt Alert	§6.6
27	High Temperature Alert	§6.6

Test	Description	Section
28	Duress Alarm	§6.6
29	Backlight: Flash 2 Times	§6.7
30	Backlight: ON	§6.7
31	Backlight: OFF	§6.7
32	Get Transaction Log	§6.8
33	Get Alert History	§6.8
34	Get Battery Level	§6.8
35	Get Wi-Fi RSSI	§6.8
36	Get Firmware Version	§6.8
37	Set Auto-Lock Timer	§6.9
38	Enable/Disable Auto-Lock	§6.9
39	Set Wrong-Attempt Lockout	§6.9
40	Enable/Disable Scramble PIN	§6.9
41	Set Volume Level	§6.9
42	Set Language	§6.9
43	Get Hardware ID List	§6.9
44	Factory Reset	§6.10
45	Reboot Device	§6.10
46	Get Device Info	§6.10
47	Export Logs	§6.10
48	Version Cycling	§6.11
49	OTA Update Check	§6.11
50	Triple-Click Boot Detection	§6.11

Appendix A. Reference Test Environment Record

For each validation cycle against this specification, the following environment details should be captured for traceability.

Table 37 — Test Environment Fields

N8 board ID: _____
N8 firmware version: _____
Wi-Fi SSID used: _____
N8 IP address: _____
MQTT broker reachable (Y/N): _____
Date of testing: _____
Tester name: _____

Table 38 — Issue Log Template

Issue ID	Test #	Description	Severity	Status

Appendix B. Approvals

This specification is subject to review and sign-off by the following roles prior to release.

Table 39 — Approval Record

Role	Name	Date
Test Engineer		
Hardware Lead		
Firmware Lead		
Backend / Dashboard Lead		